LAB	PRACTICALS
27	**Practice Lab to implement advanced JOIN operations in SQL**

Create the following tables and perform the following queries:

1. Display student names with the courses they enrolled in.

```
SELECT s.Name, c.CourseName
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
JOIN Course c ON e.CourseID = c.CourseID;
```

2. Display students and their marks.

```
SELECT s.Name, e.Marks
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID;
```

3. Display students who enrolled in DBMS.

```
SELECT s.Name
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
JOIN Course c ON e.CourseID = c.CourseID
WHERE c.CourseName = 'DBMS';
```

4. Display all students and their enrolments (include non-enrolled students).

```
SELECT s.Name, c.CourseName
FROM StudentAcademicData s
LEFT JOIN Enrolments e ON s.StuID = e.StuID
LEFT JOIN Course c ON e.CourseID = c.CourseID;
```

5. Display courses that have no enrolments.

```
SELECT c.CourseName
FROM Course c
LEFT JOIN Enrolments e ON c.CourseID = e.CourseID
WHERE e.CourseID IS NULL;
```

6. Display students with city and course credits.

```
SELECT s.Name, s.City, c.Credits
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
JOIN Course c ON e.CourseID = c.CourseID;
```

7. Display total number of courses each student has enrolled in.

```
SELECT s.Name, COUNT(e.CourseID) AS TotalCourses
FROM StudentAcademicData s
LEFT JOIN Enrolments e ON s.StuID = e.StuID
GROUP BY s.Name;
```

8. Display student names with marks greater than 80.

```
SELECT DISTINCT s.Name
```

```
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
WHERE e.Marks > 80;
```

9. Display students and courses in which credits = 4.

```
SELECT s.Name, c.CourseName
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
JOIN Course c ON e.CourseID = c.CourseID
WHERE c.Credits = 4;
```

10. Display average marks of each student.

```
SELECT s.Name, AVG(e.Marks) AS AvgMarks
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
GROUP BY s.Name;
```

11. Display highest marks obtained in each course.

```
SELECT c.CourseName, MAX(e.Marks) AS HighestMarks
FROM Course c
JOIN Enrolments e ON c.CourseID = e.CourseID
GROUP BY c.CourseName;
```

12. Display students who scored below 60 in any course.

```
SELECT DISTINCT s.Name
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
WHERE e.Marks < 60;
```

13. Display students and courses from Rajkot city only.

```
SELECT s.Name, c.CourseName
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
JOIN Course c ON e.CourseID = c.CourseID
WHERE s.City = 'Rajkot';
```

14. Display total marks gained by each student.

```
SELECT s.Name, SUM(e.Marks) AS TotalMarks
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
GROUP BY s.Name;
```

15. Display the list of students who have taken at least 3 courses.

```
SELECT s.Name
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
GROUP BY s.Name
HAVING COUNT(e.CourseID) >= 3;
```

16. Display students who have the highest marks in their courses.

```
SELECT s.Name, c.CourseName, e.Marks
FROM Enrolments e
JOIN StudentAcademicData s ON e.StuID = s.StuID
JOIN Course c ON e.CourseID = c.CourseID
WHERE e.Marks = ( SELECT MAX(Marks) FROM Enrolments
                  WHERE CourseID = e.CourseID
                );
```

17. Display students who scored above the average marks of that course.

```
SELECT s.Name, c.CourseName, e.Marks
FROM Enrolments e
JOIN StudentAcademicData s ON e.StuID = s.StuID
JOIN Course c ON e.CourseID = c.CourseID
WHERE e.Marks > ( SELECT AVG(Marks) FROM Enrolments
                  WHERE CourseID = e.CourseID
                );
```

18. Display each student's highest and lowest marks with course names.

```
SELECT s.Name, c.CourseName, e.Marks
FROM Enrolments e
JOIN StudentAcademicData s ON e.StuID = s.StuID
JOIN Course c ON e.CourseID = c.CourseID
WHERE e.Marks =
     (
       SELECT MAX(Marks) FROM Enrolments WHERE StuID = e.StuID
      )
OR    e.Marks =
     (
       SELECT MIN(Marks) FROM Enrolments WHERE StuID = e.StuID
      );
```

19. Display students enrolled in at least one 4-credit course.

```
SELECT DISTINCT s.Name
FROM StudentAcademicData s
JOIN Enrolments e ON s.StuID = e.StuID
JOIN Course c ON e.CourseID = c.CourseID
WHERE c.Credits = 4;
```

20. Display the total marks of each course and arrange them from highest to lowest.

```
SELECT c.CourseName, SUM(e.Marks) AS TotalMarks
FROM Course c
JOIN Enrolments e ON c.CourseID = e.CourseID
GROUP BY c.CourseName
ORDER BY TotalMarks DESC;
```